

GROUP 3: SENSOR DATA PROCESSING SYSTEM

1. OVERVIEW

This project implements a Sensor Data Processing System designed for agricultural monitoring. The same system is to be developed across four programming languages to demonstrate that the underlying computational logic remains consistent regardless of implementation language.

The focus is not on syntax differences, but on how a real-world agricultural problem can be translated into a structured computational system and progressively improved using programming concepts.

The system is developed progressively, where each milestone builds directly on the previous one. No part of the system is discarded; instead, it is refined and extended.

2. PROBLEM DEFINITION

Modern farming increasingly depends on accurate and continuous monitoring of environmental conditions such as soil moisture, temperature, and rainfall. Manual monitoring is inefficient, especially when dealing with large-scale farms or multiple fields.

This project models a system that simulates how farm sensors operate. Instead of using real hardware, the system processes predefined or user-provided data to mimic sensor readings.

The objective is to:

- Process environmental data
- Interpret conditions affecting crops
- Generate meaningful decisions such as irrigation recommendations or warnings

The system is designed to evolve from a simple single-sensor model into a scalable multi-sensor simulation.

3. REAL WORLD TO SYSTEM MAPPING

Each variable in the program represents a real agricultural parameter:

- **moisture_level** → soil water content (0.0 to 1.0 scale)
- **temperature** → ambient air temperature (°C)
- **rainfall** → amount of rainfall (mm)
- **sensor_id** → identifier for each simulated sensor
- **status** → indicates whether the sensor is ACTIVE or FAULTY

Moisture Classification:

- < 0.2 → DRY (critical condition)
- 0.2 – 0.4 → LOW (irrigation needed soon)
- 0.4 – 0.7 → OPTIMAL (healthy conditions)
- 0.7 → HIGH (risk of waterlogging)

Temperature Condition:

- 35°C → High temperature warning

This mapping ensures that the system reflects realistic agricultural conditions.

4. SYSTEM STRUCTURE

The system follows a simple processing pipeline:

Input → Processing → Decision → Output

- Input: sensor readings (simulated values)
- Processing: calculations and classification
- Decision: determine actions (e.g., irrigation)
- Output: display results to the user

5. MILESTONE 1

Computational Foundations

Objective

To translate agricultural sensor behavior into a working computational model using basic programming constructs.

Implementation

At this stage, the system is implemented using a procedural approach. It focuses on handling data and applying logic directly without structural abstraction.

The system performs the following:

- Initializes variables representing environmental conditions
- Stores multiple moisture readings in a dataset
- Computes:
 - Total moisture
 - Average moisture
 - Maximum and minimum values
- Iterates through readings using loops
- Classifies each reading based on defined thresholds
- Produces simple decisions:
 - Irrigation recommendation when moisture is low
 - Temperature warning when values exceed limits

Key Insight

This milestone establishes the **foundation of the system logic**. All future development builds on these computations and decisions.

6. MILESTONE 2

Milestone 2 builds directly on the computational logic developed in Milestone 1 by restructuring the system using object-oriented principles.

Control Logic & Object-Oriented Design

Objective

To restructure the system using object-oriented principles for better organization, scalability, and realism.

Implementation

The system is redesigned around a **Sensor class**, which represents an individual field sensor.

Each sensor object contains:

- State (moisture, temperature, status)
- Behavior (methods for validation and processing)

Core Features

- **Encapsulation**
Sensor data is stored within the class and accessed through methods, ensuring controlled interaction.
- **Validation Logic**
Sensor readings are checked before processing:
 - Moisture must be within 0.0–1.0
 - Temperature must be within realistic limitsInvalid readings result in the sensor being marked as FAULTY.
- **Processing Methods**
Each sensor:
 - Classifies its moisture level
 - Determines required actions (e.g., irrigation)
- **Multi-Sensor Simulation**
Multiple sensor objects are created and processed using loops, simulating a real farm environment.

System Evolution

Aspect	Milestone 1	Milestone 2
Structure	Procedural	Object-Oriented
Data Handling	Separate variables	Encapsulated in class
Sensors	Single	Multiple objects
Validation	None	Implemented
Logic	Direct	Organized in methods

Key Insight

This milestone transforms the system from a simple script into a modular and scalable model, closer to real engineering software.

7. DESIGN CONSISTENCY ACROSS LANGUAGES

The system is to be implemented in Python, Java, C++, and Smalltalk. Despite differences in syntax, the following remain identical:

- System logic
- Data flow
- Decision rules
- Class structure

This ensures that the project demonstrates understanding of concepts, not just programming syntax.

8. GROUP IMPLEMENTATION APPROACH

The group is organized into pairs, with each pair responsible for one programming language implementation.

All pairs:

- Follow the same system design
- Maintain consistent logic and structure
- Contribute to a unified final system

This approach ensures both collaboration and consistency across the project.

9. CONCLUSION

The Sensor Data Processing System demonstrates how real-world agricultural monitoring can be modeled computationally.

Milestone 1 established the core logic using basic programming constructs.

Milestone 2 introduced structure and scalability through object-oriented design.

The system is now prepared for further development in subsequent milestones, including data persistence, concurrency, and user interface integration.

The current system provides a strong foundation that will support further extensions such as modular architecture, data persistence, and concurrent processing in later milestones.